Panorama Image Stitching Using Feature-Based Alignment

COE843 – Intro to Computer Vision

Name: Bilal Mahmud, Chanuth Pathirana, Leo Marion-Landais Lorenzo

Course: COE 843

Group: 65

Department of Computer and Electrical Engineering

Instructor: Richard Wang

Abstract

The algorithm that is being presented stitches images with similar features together to create a final panoramic image that contains all the images blended together. The algorithm used to implement this uses the SIFT, FLANN and RANSAC algorithms to find keypoints, match those keypoints of the images and map the coordinates to one image set to another within a certain error. The implemented method was compared to another method using Harris corner detection and least squares method. It was theorized that the method using the SIFT, FLANN and RANSAC algorithms in Python OpenCV would perform more efficiently than this algorithm, with the only computationally demanding section being the gradient calculation in the SIFT algorithm. Overall, the method was able to be successfully implemented and tested, with visualizations of each individual algorithms' effects being put in place. Source code for the project is : https://drive.google.com/file/d/1TjvSxIwz-mV9-9KI-CcitQGdeCRhS8N9/view?usp=sharing

Introduction

Image Stitching is one of the most useful concepts in computer vision. It involves taking separate images and combining them to create a single large scale image that is cohesive and represents a cohesive overall structure. For example, multiple separate images of a snowy mountainside can be stitched together to create a panoramic image. A panoramic image is usually much larger than a normal image and can either be very wide in terms of length or very tall in terms of height. These types of images are very difficult to create without the use of image stitching, which is why the procedure is used very commonly when it comes to creating panoramic images. Previous studies regarding this topic have involved computing the homography matrix from one image to another using the least squares method and determining the corresponding points. Then it maps the points to the original image using inverse wrapping (inverse homography). Finally, it overlays the second image to the first using

those corresponding points to stitch the image together. To automatically choose the corresponding points, Harris corner section along with Random Sample Consensus (RANSAC) is used, along with a number of other features (Automatic Image Stitching, n.d).

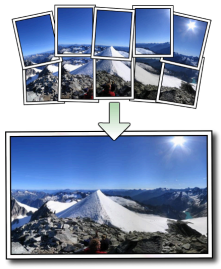

Figure 1: The process of Image Stitching to create a Panorama. (Sood, 2019)

This method is very simple and fully automates the stitching. The problem with this method is that the inverse warping and the Harris corner detection can both be computationally intensive to the hardware resulting in lower run times. In the following approach, Scale-Invariant Feature Transform (SIFT) and Fast Library for Approximate Nearest Neighbors (FLANN) will be used along with RANSAC. SIFT will first determine the features of the images, and FLANN will find what sections of the images map to each other using the SIFT output. RANSAC will then be used for Homography estimation, and finally the images will be stitched together. This process is much faster, as using the Harris corner detection algorithm is very computationally intensive (GeeksforGeeks, 2025). This algorithm will be developed in Python using the OpenCV library. This library has functions that relate closely to feature detection and image filtering.

Technical Method

As discussed in the previous section, the implemented method uses these steps:

1. Use SIFT to detect keypoints and feature descriptions in the images
2. Use FLANN and Lowe's Ratio Test to detect corresponding points using the SIFT keypoints
3. Compute the Homography matrix H and remove outliers using RANSAC
4. Use the Homography Matrix to map coordinates in one image to another
5. Blend the two images together

First, the SIFT algorithm is used to detect keypoints and describe the overall features of the image. This is used to find what sections of one image map to points on another image. The image switching process will begin using this information. In this process, the input image is repeatedly blurred using a gaussian filter, each blur at different levels. Some images will have major blur, others will only have minor blur. Then 2 blurred images are subtracted to each other. This is called the Difference of Gaussians (DoG). Finally, each pixel coordinate of the DoG image will be compared to its neighbors of its own image as well as its neighbors from a higher and lower DoG scale. If that pixel is the maximum or minimum value, that pixel will be a candidate for a keypoint (Tyagi, 2019). The algorithm will make sure that some keypoints will be filtered out due to noise.

For detecting features, the area around each keypoint is taken and divided into regions. A gradient of each region is calculated. Each region's 8 bin orientation histogram will then be calculated containing the gradient magnitudes (Tyagi, 2019). Each histogram is then combined, and the vector of all the values in the combined histogram is the description of the individual section of the image. The advantage to SIFT is that it is invariant to the scale and the rotation of the image. This is very important when it comes

to image stitching, as two images could be oriented differently in terms of rotation and scale.

Because the algorithm is using gradients, it is more computationally intensive than other algorithms, but makes up for it in terms of flexibility in how it can detect key points to images under rotation and zoom.

Next, the FLANN algorithm along with Lowe's Ratio Test is performed to find the corresponding points of the images. It finds the keypoint in the second image that has the most similarity to the keypoint in the first image. FLANN first creates a tree, and the descriptors are sorted using that spanning tree by calculating the median of all the keypoints that belong to that node. So when finding the most similar keypoint in the second image to the first, the first images' keypoint traverses through the spanning tree until it lands at the final node. Lowe's Ratio test then filters out some keypoint matches. It uses the ratio of the distance to the similar keypoint descriptor of both images. If this ratio is below a certain threshold (can be manually set), then the match is saved. If it is above the threshold, it is discarded (OpenCV, n.d). The combination of these algorithms makes it so two corresponding points can be found very quickly. It is a very efficient and an algorithm with a low runtime, although it is prone to some inaccuracies

Homography H is essential for translating points in one image to another using the corresponding points. The formula for this is:

$$\begin{bmatrix} x' \\ y' \\ 1 \end{bmatrix} = \begin{bmatrix} h_{11} & h_{12} & h_{13} \\ h_{21} & h_{22} & h_{23} \\ h_{31} & h_{32} & h_{33} \end{bmatrix} \begin{bmatrix} x \\ y \\ 1 \end{bmatrix}$$

A random set of 4 corresponding points from each of the 2 images are used. When a translation is performed, the point may not match up exactly in the second images' coordinate system compared to the first image. An error distance is calculated using the homography coordinate and the actual mapped coordinate. If the error is below a certain threshold, this is called an inliner. Above the threshold means it's an outlier, and will be discarded. This is performed multiple times for accuracy (GeeksforGeeks, 2025) .

After this is done, the images are finally blended together to form the combined image. This process is repeated again to blend the combined image to a third image and so on.

Experimental Results

The following images were taken from the inside of a house of one of our group members for this project. Here are the results for the implemented algorithm. It will contain a visual representation of the 3 original images, detecting keypoints, the matches that have been found, the inliners and the final panorama:

Figures 2, 3 and 4: The original images

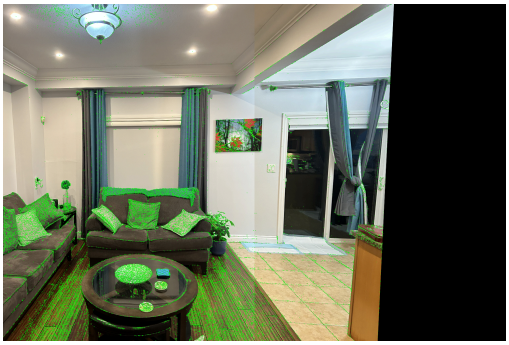

Figure 5: Finding the keypoints using SIFT

Figure 6: Comparing the images using FLANN and finding the matches

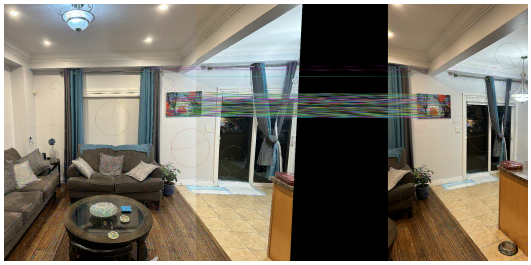

Figure 7:Using the Homography H to map coordinates and finding the inliners using RANSAC

Figure 8: Final Panorama using image stitching

The original images are represented in Figures 2, 3 and 4. Figure 5 represents capturing the keypoints of the Image. The green dots represent a keypoint. Figure 6 contains the feature matching using FLANN. The lines show the features that are mapped to each other in the different images. Figure 7 shows the using the Homography H to map the coordinates from one image set to another and finding the inliners. The circles with lines show the error. A bigger circle means there is a large amount of error when mapping the coordinates. Finally, Figure 8 shows the final panorama after the images have been blended together.

Conclusion

Overall, the original images in the previous section were able to be transformed by implementing SIFT for keypoint detecting, FLANN for matching the features, Homography H and RANSAC for mapping an image to another and keeping the inliners, and blending the images together.

This algorithm performs more efficiently than the previous image stitching method. Additionally, the SIFT section of the overall algorithm is the only section that has a relatively high computational expense compared to the other sections of this algorithm. Using SIFT allows for very high scalability in terms of the images being in rotation and zooming, so this additional computational expense means that the algorithm is much more robust. The implemented solution shows that using Python's OpenCV library, which contains SIFT, FLANN and coordinates mapping among other features, garners much more efficient results than brute force methods such as using Harris corner detection.

For the individual contributions of this implemented method, Leo Marion Landais Lorenzo mainly implemented the algorithms and did the early coding. Chanuth Pathirana mostly collected the data, tested and debugged the algorithm and analyzed the results. Bilal Mahmud mainly covered the overall background information regarding the implemented method, documented the results and did the report writing.

References

Sood, N. (2019, July 31). *Image Stitching to create a Panorama*. https://medium.com/@navekshasood/image-stitching-to-create-a-panorama-5e030ecc8f7

Tyagi, D. (2019, March 16). *Introduction to SIFT(Scale Invariant Feature Transform)*. https://medium.com/@deepanshut041/introduction-to-sift-scale-invariant-feature-transform-65d7f3a72d40

OpenCV: Feature Matching with FLANN. (n.d). Opencv.org. https://docs.opencv.org/3.4/d5/d6f/tutorial_feature_flann_matcher.html

GeeksforGeeks. (2025, July 23). *What is Homography? How to estimate homography between two images?* GeeksforGeeks. https://www.geeksforgeeks.org/computer-vision/what-is-homography-how-to-estimate-homography-between-two-images/

Project #4: Automatic Image Stitching. (n.d). Ulaval.ca. https://vision.gel.ulaval.ca/~jflalonde/cours/4105/h14/tps/results/tp4/raziehtoony/index.html

GeeksforGeeks. (2025, August 5). *Python | Corner detection with Harris Corner Detection method using OpenCV*. https://www.geeksforgeeks.org/python/python-corner-detection-with-harris-corner-detection-method-using-opencv/